Name: Mrigaj Shaw
Roll No: 2024CSB041

Assignment 4

Problem 1

Problem Statement: Write a C program to represent and analyze two real-world networks using **graph data structures**, where one graph is **sparse** and the other is **dense**. Let the number of vertices be $n = 100$. For the sparse graph, take the number of edges as $e_s = 120$, such that

$$e_s \ll \frac{n(n-1)}{2},$$

and generate the edges **randomly**. Represent this sparse graph using an **adjacency list** for both **unweighted edges** (showing only connections) and **weighted edges** (assigning a random cost or distance to each edge).

For the dense graph, take the number of edges as $e_d = 4500$, such that

$$e_d \approx \frac{n(n-1)}{2},$$

and generate the edges **randomly**. Represent this dense graph using an **adjacency matrix** for both **unweighted and weighted edges**. The program should **automatically generate random vertex pairs and edge weights**, construct the corresponding graph representations, and **display the adjacency list and adjacency matrix**.

Code

```c
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct Node {
    struct Node *next;
    int vertex;
    int weight;
} Node;

Node **createAdjacencyList(int vertices) {
    Node **list = (Node **)malloc(vertices * sizeof(Node *));
    for (int i = 0; i < vertices; i++) {
        list[i] = NULL;
    }
    return list;
}

bool addEdgeAdjacencyListDirectedWithoutWeight(Node **list, int u, int v) {
    if (list == NULL)
        return false;
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = 0;
    newNode->next = list[u];
    list[u] = newNode;

    return true;
}

```

```c
bool addEdgeAdjacencyListUnDirectedWithoutWeight(Node **list, int u, int v) {
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, u, v))
        return false;
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, v, u))
        return false;
    return true;
}

bool addEdgeAdjacencyListDirectedWithWeight(Node **list, int u, int v,
                                            int weight) {
    if (list == NULL)
        return false;

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = weight;
    newNode->next = list[u];
    list[u] = newNode;

    return true;
}

bool addEdgeAdjacencyListUnDirectedWithWeight(Node **list, int u, int v,
                                              int weight) {
    if (!addEdgeAdjacencyListDirectedWithWeight(list, u, v, weight))
        return false;
    if (!addEdgeAdjacencyListDirectedWithWeight(list, v, u, weight))
        return false;
    return true;
}

void displayAdjacencyList(Node **list, int vertices) {
    printf("\nAdjacency List Representation\n");
    printf("------------------------------\n");
    for (int i = 0; i < vertices; i++) {
        printf("Vertex [%d]: ", i);
        Node *temp = list[i];

        while (temp != NULL) {
            printf("%d(w:%d) -> ", temp->vertex, temp->weight);
            temp = temp->next;
        }
        printf("NULL\n");
    }
    printf("------------------------------------\n");
}

void freeAdjacencyList(Node **list, int vertices) {
    if (list == NULL)
        return;

    for (int i = 0; i < vertices; i++) {
        Node *current = list[i];
        while (current != NULL) {
            Node *temp = current;
            current = current->next;
            free(temp);
```

```c
        }
    }
    free(list);
}

void displayAdjacencyMatrix(int matrix[100][100]) {
    int n = 100;
    int limit = 15;
    printf("\nAdjacency Matrix (Showing top %dx%d for readability)\n", limit,
           limit);
    printf("--------------------------------------------------\n");

    // Print column headers
    printf("     ");
    for (int i = 0; i < limit; i++) {
        printf("%2d ", i);
    }
    printf("...\n");
    for (int i = 0; i < limit; i++) {
        printf("%3d |", i);
        for (int j = 0; j < limit; j++) {
            printf(matrix[i][j] ? " 1 " : " . ");
        }
        printf("...\n");
    }
    printf("...\n(Remaining %d rows and columns hidden)\n", n - limit);
}

int main() {
    int n = 100; // number of vertices
    // the vertices are [0, 99];
    srand(time(NULL));
    Node **list = createAdjacencyList(n);

    // Adjacency list
    for (int i = 0; i < n; i++) {
        int u = rand() % 100;
        int v = rand() % 100;
        addEdgeAdjacencyListUnDirectedWithoutWeight(list, u, v);
    }

    displayAdjacencyList(list, n);

    int edges_sparse = 120;
    for (int i = 0; i < edges_sparse; i++) {
        int u = rand() % 100;
        int v = rand() % 100;
        int weight = rand() % 50 + 1;
        addEdgeAdjacencyListUnDirectedWithWeight(list, u, v, weight);
    }

    displayAdjacencyList(list, n);

    freeAdjacencyList(list, n);

    // Adjacency Matrix
    int adjacencyMatrix[100][100] = {0}; // Fills the entire matrix with 0s
    int edges_matrix = 4500;
    for (int i = 0; i < edges_matrix; i++) {
        int u = rand() % 100;
```

```
        int v = rand() % 100;
        adjacencyMatrix[u][v] = 1;
        adjacencyMatrix[v][u] = 1;
    }
    displayAdjacencyMatrix(adjacencyMatrix);
    return 0;
}
```

Output

```
Adjacency List Representation
-----------------------------
Vertex [0]: 31(w:0) -> NULL
Vertex [1]: 21(w:0) -> 49(w:0) -> NULL
Vertex [2]: 87(w:0) -> 6(w:0) -> 54(w:0) -> NULL
Vertex [3]: NULL
Vertex [4]: 31(w:0) -> 81(w:0) -> 83(w:0) -> 8(w:0) -> NULL
Vertex [5]: 18(w:0) -> NULL
Vertex [6]: 2(w:0) -> 41(w:0) -> 40(w:0) -> NULL
Vertex [7]: 74(w:0) -> 43(w:0) -> 64(w:0) -> 75(w:0) -> NULL
Vertex [8]: 33(w:0) -> 4(w:0) -> 12(w:0) -> NULL
Vertex [9]: 17(w:0) -> 96(w:0) -> NULL
Vertex [10]: 33(w:0) -> 86(w:0) -> NULL
Vertex [11]: 26(w:0) -> 69(w:0) -> NULL
Vertex [12]: 68(w:0) -> 92(w:0) -> 60(w:0) -> 51(w:0) -> 30(w:0) -> 8(w:0) -> NULL
Vertex [13]: 84(w:0) -> NULL
Vertex [14]: 32(w:0) -> 28(w:0) -> NULL
Vertex [15]: 85(w:0) -> 65(w:0) -> 16(w:0) -> NULL
Vertex [16]: 15(w:0) -> NULL
Vertex [17]: 9(w:0) -> NULL
Vertex [18]: 36(w:0) -> 35(w:0) -> 62(w:0) -> 5(w:0) -> 86(w:0) -> 95(w:0) -> NULL
Vertex [19]: 31(w:0) -> 89(w:0) -> 91(w:0) -> NULL
```

```
Adjacency Matrix (Showing top 15x15 for readability)
-----------------------------------------------------
      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 ...
  0 | 1  .  1  .  1  1  .  .  1  1  1  1  1  1  1 ...
  1 | .  .  1  1  .  1  .  .  1  .  .  .  1  1  1 ...
  2 | 1  1  1  1  .  1  1  1  1  .  1  1  .  .  1 ...
  3 | .  1  1  .  1  1  .  .  1  1  1  .  1  .  . ...
  4 | 1  .  .  1  .  1  .  1  .  1  .  1  .  .  1 ...
  5 | 1  1  1  1  1  1  .  1  1  .  .  1  .  .  . ...
  6 | .  .  1  .  .  .  .  1  .  1  1  1  1  1  . ...
  7 | .  .  1  .  1  1  1  .  1  .  1  1  .  .  . ...
  8 | 1  1  1  1  .  1  .  1  1  .  .  1  1  1  . ...
  9 | 1  .  .  1  1  .  1  .  .  .  1  1  .  1  . ...
 10 | 1  .  1  1  .  .  1  1  .  1  1  1  1  .  . ...
 11 | 1  .  1  .  1  1  1  1  1  1  1  .  .  .  1 ...
 12 | 1  1  .  1  .  .  1  .  1  .  1  .  1  .  1 ...
 13 | 1  1  .  .  .  .  1  .  1  1  .  .  .  .  1 ...
 14 | 1  1  1  .  1  .  .  .  .  .  .  1  1  1  . ...
...
(Remaining 85 rows and columns hidden)
```

Inference

- The graph was successfully implemented using both Adjacency List and Adjacency Matrix representations.
- The **Adjacency List** requires $O(V + E)$ space, making it highly efficient for sparse graphs.
- The **Adjacency Matrix** requires $O(V^2)$ space, which is optimal for dense graphs and $O(1)$ edge-lookups, but wastes memory for sparse networks.
- *Note: The core `AdjacencyList.h` header was implemented from scratch and serves as the foundational data structure utilized across all subsequent graph problems in this assignment.*

Problem 2

Problem Statement: Write a C program to connect 6 cities (C1–C6) with minimum cost using **Prim's Algorithm** and a **Min-Priority Queue**. The cost of connecting each pair of cities (in lakhs) is:

$$\begin{array}{lll} (C1, C2) = 3, & (C1, C3) = 1, & (C1, C4) = 6, \\ (C2, C3) = 5, & (C2, C5) = 3, & (C3, C4) = 5, \\ (C3, C6) = 4, & (C4, C6) = 2, & (C5, C6) = 6 \end{array}$$

Your program should:

1. Represent the graph using an **adjacency list**.
2. Implement **Prim's Algorithm** starting from vertex C1 using a **Min-Priority Queue**.
3. Display the edges in the **Minimum Spanning Tree (MST)**, the **total minimum cost**, and the **order of vertices added**.

Code

```c
#include "../AdjacencyList/AdjacencyList.h"
#include "../MinHeap/MinHeap.h"
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 6; // 6 cities (C1 to C6)
    Node **adjacencyList = createAdjacencyList(n);

    // mapping C1->0, C2->1, C3->2, C4->3, C5->4, C6->5 to make indexing easier
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 1, 3);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 2, 1);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 3, 6);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 1, 2, 5);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 1, 4, 3);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 2, 3, 5);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 2, 5, 4);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 3, 5, 2);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 4, 5, 6);

    // Prim's Algo setup
    bool visited[6] = {false};
    int total_cost = 0;
    int edges_added = 0;

    // array to track the order of cities we visit
    int order[6];
    int order_idx = 0;

    // queue size is large enough to hold all possible edges
    MinHeap *pq = createMinHeap(100);

    // Start from C1 (which is index 0)
    visited[0] = true;
    order[order_idx++] = 0;

```

```c
    // throw all initial edges from C1 into the priority queue
    Node *temp = adjacencyList[0];
    while (temp != NULL) {
        insertMinHeap(pq, 0, temp->vertex, temp->weight);
        temp = temp->next;
    }

    printf("\nEdges in the Minimum Spanning Tree (MST):\n");
    printf("----------------------------------------\n");

    // run until we connect all V-1 edges (6 cities = 5 edges)
    while (!isEmpty(pq) && edges_added < n - 1) {
        Edge minEdge = extractMin(pq);

        // if the destination city is already in the MST, skip it
        if (visited[minEdge.v]) {
            continue;
        }

        // Add edge to our MST
        visited[minEdge.v] = true;
        total_cost += minEdge.weight;
        edges_added++;
        order[order_idx++] = minEdge.v; // record the order

        printf("C%d - C%d \t Cost: %d lakhs\n", minEdge.u + 1, minEdge.v + 1,
               minEdge.weight);

        // push all adjacent edges of the new city into the heap
        temp = adjacencyList[minEdge.v];
        while (temp != NULL) {
            if (!visited[temp->vertex]) {
                insertMinHeap(pq, minEdge.v, temp->vertex, temp->weight);
            }
            temp = temp->next;
        }
    }

    printf("----------------------------------------\n");
    printf("Total Minimum Cost: %d lakhs\n\n", total_cost);

    printf("Order of vertices added: ");
    for (int i = 0; i < n; i++) {
        printf("C%d", order[i] + 1); // +1 because we mapped 0 to C1
        if (i < n - 1)
            printf(" -> ");
    }
    printf("\n");

    freeMinHeap(pq);
    freeAdjacencyList(adjacencyList, n);

    return 0;
}
```

Output

```
Edges in the Minimum Spanning Tree (MST):
-----------------------------------------
C1 - C3          Cost: 1 lakhs
C1 - C2          Cost: 3 lakhs
C2 - C5          Cost: 3 lakhs
C3 - C6          Cost: 4 lakhs
C6 - C4          Cost: 2 lakhs
-----------------------------------------
Total Minimum Cost: 13 lakhs

Order of vertices added: C1 -> C3 -> C2 -> C5 -> C6 -> C4
```

Inference

- Prim's algorithm was implemented using the Adjacency List and a custom `MinHeap.h` Min-Priority Queue (see Appendix).
- The overall time complexity is $O(E \log V)$ because extracting the minimum weight edge from the heap takes $O(\log V)$ time, performed for up to E edges in the worst case.
- A boolean `visited` array was utilized alongside a "lazy" heap approach.
- This efficiently ignores stale edges and prevents the formation of cycles during the Minimum Spanning Tree (MST) construction.

Problem 3

Problem Statement: Write a C program to connect 6 cities (C1–C6) with minimum cost using **Kruskal's Algorithm**. The cost of connecting each pair of cities (in lakhs) is:

$$\begin{array}{lll} (C1, C2) = 3, & (C1, C3) = 1, & (C1, C4) = 6, \\ (C2, C3) = 5, & (C2, C5) = 3, & (C3, C4) = 5, \\ (C3, C6) = 4, & (C4, C6) = 2, & (C5, C6) = 6 \end{array}$$

Your program should:

1. Represent the graph using an **edge list**.
2. Sort all edges in ascending order and construct the **Minimum Spanning Tree (MST)** using **Union-Find (Disjoint Set)** to prevent cycles.
3. Display the edges included in the MST, the **total minimum cost**, and the **order in which edges are added**.

Code

```c
#include "../../Assignment_5/DisjointSetUnion/DisjointSetUnion.h"
#include "../AdjacencyList/AdjacencyList.h"
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u;
    int v;
    int weight;
} GraphEdge;

// helper for qsort to sort edges by weight ascending
int compareEdges(const void *a, const void *b) {
    GraphEdge *edgeA = (GraphEdge *)a;
    GraphEdge *edgeB = (GraphEdge *)b;
    return edgeA->weight - edgeB->weight;
}

int main() {
    int V = 6; // map C1-C6 to indices 0-5

    Node **adjacencyList = createAdjacencyList(V);

    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 1, 3);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 2, 1);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 0, 3, 6);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 1, 2, 5);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 1, 4, 3);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 2, 3, 5);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 2, 5, 4);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 3, 5, 2);
    addEdgeAdjacencyListUnDirectedWithWeight(adjacencyList, 4, 5, 6);

    // convert adjacency list to a flat edge array for kruskal's
    int max_edges = V * (V - 1) / 2;
    GraphEdge *edges = (GraphEdge *)malloc(max_edges * sizeof(GraphEdge));
    int edge_count = 0;
```

```c

    for (int i = 0; i < V; i++) {
        Node *temp = adjacencyList[i];
        while (temp != NULL) {
            // i < vertex check stops us from adding reverse edges (like 1->0)
            // twice
            if (i < temp->vertex) {
                edges[edge_count].u = i;
                edges[edge_count].v = temp->vertex;
                edges[edge_count].weight = temp->weight;
                edge_count++;
            }
            temp = temp->next;
        }
    }

    qsort(edges, edge_count, sizeof(GraphEdge), compareEdges);

    DisjointSet *ds = createSet(V);

    int mst_edges_count = 0;
    int current_edge_idx = 0;
    int total_cost = 0;

    printf("\nEdges included in the Minimum Spanning Tree (MST):\n");
    printf("--------------------------------------------------\n");

    // stop when we have exactly V-1 edges connected
    while (mst_edges_count < V - 1 && current_edge_idx < edge_count) {
        GraphEdge next_edge = edges[current_edge_idx++];

        int rootU = find(ds, next_edge.u);
        int rootV = find(ds, next_edge.v);

        // only add if they don't share a parent (prevents cycles)
        if (rootU != rootV) {
            mst_edges_count++;
            total_cost += next_edge.weight;

            unionSets(ds, next_edge.u, next_edge.v);

            // +1 to convert indices back to C1, C2 etc.
            printf("Edge: C%d - C%d \t Cost: %d lakhs\n", next_edge.u + 1,
                   next_edge.v + 1, next_edge.weight);
        }
    }

    printf("--------------------------------------------------\n");
    printf("Total Minimum Cost: %d lakhs\n\n", total_cost);

    // cleanup memory
    freeSet(ds);
    free(edges);
    freeAdjacencyList(adjacencyList, V);

    return 0;
}
```

Output

```
Edges included in the Minimum Spanning Tree (MST):
--------------------------------------------------
Edge: C1 - C3    Cost: 1 lakhs
Edge: C4 - C6    Cost: 2 lakhs
Edge: C2 - C5    Cost: 3 lakhs
Edge: C1 - C2    Cost: 3 lakhs
Edge: C3 - C6    Cost: 4 lakhs
--------------------------------------------------
Total Minimum Cost: 13 lakhs
```

Inference

- Kruskal's algorithm was implemented by extracting a flat array of unique edges from the undirected Adjacency List.
- Edges were sorted in ascending order using Quicksort, dictating the overall $O(E \log E)$ time complexity.
- Cycle detection was handled efficiently using a custom `DisjointSetUnion.h` (see Appendix) featuring both path compression and union by rank.
- The near $O(1)$ time complexity of these Union-Find operations ensures they do not bottleneck the primary sorting phase.

Problem 4

Problem Statement: Write a C program to compute the **Minimum Spanning Tree (MST)** of a large-scale **smart city power distribution network** consisting of two independent weighted graphs $G_1(V_1, E_1)$ and $G_2(V_2, E_2)$, interconnected by a **single high-capacity transmission edge** having the **maximum weight among all edges**. This high-weight edge must always be included in the final MST. Apply **Kruskal's algorithm** on graph G_1, considering it as a **sparse graph**, and apply **Prim's algorithm** on graph G_2, considering it as a **dense graph**. After computing the MSTs of both graphs separately, merge them using the **mandatory high-weight edge** to construct the **global MST** of the combined network. The program should take the number of vertices, edges, and weighted edge lists as input, compute the **final MST**, display the **selected edges**, and output the **total cost** of the spanning tree.

Code

```c
#include "../../Assignment_5/DisjointSetUnion/DisjointSetUnion.h"
#include "../AdjacencyList/AdjacencyList.h"
#include "../MinHeap/MinHeap.h"
#include <stdio.h>
#include <stdlib.h>

// struct just for G1 edge list
typedef struct {
    int u, v, weight;
} GraphEdge;

// sort helper for qsort (ascending weight)
int compareEdges(const void *a, const void *b) {
    return ((GraphEdge *)a)->weight - ((GraphEdge *)b)->weight;
}

int main() {
    int V1, E1, V2, E2;

    // input G1 as sparse (using flat array for kruskal's)
    printf("Enter V1 and E1 for Graph G1: ");
    scanf("%d %d", &V1, &E1);

    GraphEdge *edgesG1 = (GraphEdge *)malloc(E1 * sizeof(GraphEdge));
    printf("Enter %d edges for G1 (u v weight):\n", E1);
    for (int i = 0; i < E1; i++) {
        scanf("%d %d %d", &edgesG1[i].u, &edgesG1[i].v, &edgesG1[i].weight);
    }

    // input G2 as dense (using adjacency list for prim's)
    printf("\nEnter V2 and E2 for Graph G2: ");
    scanf("%d %d", &V2, &E2);

    Node **adjListG2 = createAdjacencyList(V2);
    printf("Enter %d edges for G2 (u v weight):\n", E2);
    for (int i = 0; i < E2; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        addEdgeAdjacencyListUnDirectedWithWeight(adjListG2, u, v, w);
    }

    // mandatory bridge edge
    int connect_u, connect_v, connect_weight;
```

```c
    printf("\nEnter mandatory connecting edge (u_in_G1 v_in_G2 weight): ");
    scanf("%d %d %d", &connect_u, &connect_v, &connect_weight);

    int total_cost = 0;
    printf("\n==================================================\n");
    printf("        GLOBAL MINIMUM SPANNING TREE (MST)\n");
    printf("==================================================\n");

    // Phase 1: Kruskal's on G1
    printf("\n[Graph G1 - Kruskal's MST]\n");
    qsort(edgesG1, E1, sizeof(GraphEdge), compareEdges);
    DisjointSet *ds = createSet(V1);
    int mst1_edges = 0;

    for (int i = 0; i < E1 && mst1_edges < V1 - 1; i++) {
        int rootU = find(ds, edgesG1[i].u);
        int rootV = find(ds, edgesG1[i].v);

        // if roots differ, no cycle
        if (rootU != rootV) {
            unionSets(ds, edgesG1[i].u, edgesG1[i].v);
            printf("G1 Edge: %2d - %2d \t Cost: %d\n", edgesG1[i].u,
                   edgesG1[i].v, edgesG1[i].weight);
            total_cost += edgesG1[i].weight;
            mst1_edges++;
        }
    }

    // Phase 2: Prim's on G2
    printf("\n[Graph G2 - Prim's MST]\n");
    bool *visited = (bool *)calloc(V2, sizeof(bool));

    // allocate extra space for dense graph edges
    MinHeap *pq = createMinHeap(E2 * 2);
    int mst2_edges = 0;

    // start from vertex 0
    visited[0] = true;
    Node *temp = adjListG2[0];
    while (temp != NULL) {
        insertMinHeap(pq, 0, temp->vertex, temp->weight);
        temp = temp->next;
    }

    while (!isEmpty(pq) && mst2_edges < V2 - 1) {
        Edge minEdge = extractMin(pq);

        if (visited[minEdge.v])
            continue; // cycle check

        visited[minEdge.v] = true;
        printf("G2 Edge: %2d - %2d \t Cost: %d\n", minEdge.u, minEdge.v,
               minEdge.weight);
        total_cost += minEdge.weight;
        mst2_edges++;

        // push unvisited neighbors
        temp = adjListG2[minEdge.v];
        while (temp != NULL) {
            if (!visited[temp->vertex]) {
```

```
                insertMinHeap(pq, minEdge.v, temp->vertex, temp->weight);
            }
            temp = temp->next;
        }
    }

    // Phase 3: Combine with mandatory edge
    printf("\n[Interconnecting Transmission Edge]\n");
    printf("Link: G1(%d) - G2(%d) \t Cost: %d\n", connect_u, connect_v,
           connect_weight);
    total_cost += connect_weight;

    printf("--------------------------------------------------\n");
    printf("Total Cost of Global MST: %d\n\n", total_cost);

    // cleanup
    freeSet(ds);
    free(edgesG1);
    free(visited);
    freeMinHeap(pq);
    freeAdjacencyList(adjListG2, V2);

    return 0;
}
```

Output

```
==================================================
       GLOBAL MINIMUM SPANNING TREE (MST)
==================================================

[Graph G1 - Kruskal's MST]
G1 Edge:  0 -  1          Cost: 3
G1 Edge:  1 -  2          Cost: 4
G1 Edge:  2 -  3          Cost: 5

[Graph G2 - Prim's MST]
G2 Edge:  0 -  1          Cost: 3
G2 Edge:  1 -  2          Cost: 4
G2 Edge:  2 -  3          Cost: 5

[Interconnecting Transmission Edge]
Link: G1(2) - G2(3)       Cost: 10
--------------------------------------------------
Total Cost of Global MST: 34
```

Inference

- This hybrid problem effectively leverages the distinct architectural strengths of both MST algorithms.

- **Kruskal's algorithm** is optimally applied to the sparse graph (G_1) as sorting a small edge list is highly efficient ($O(E_1 \log E_1)$).
- **Prim's algorithm** is applied to the dense graph (G_2), where its $O(E_2 \log V_2)$ complexity natively handles heavy interconnections without requiring a massive global array sort.
- Connecting the two independently generated MSTs via the single mandatory high-capacity transmission edge acts as a bridge.
- This final connection takes $O(1)$ time and mathematically guarantees a valid, globally optimal MST for the combined smart city network.

Appendix: Shared Data Structures

The following header files were implemented from scratch and utilized across multiple problems in this assignment to handle graph and priority operations efficiently.

1. Min-Priority Queue (`MinHeap.h`)

```c
#ifndef MIN_HEAP_H
#define MIN_HEAP_H

#include <stdbool.h>
#include <stdlib.h>

// structure to hold graph edges for the priority queue
typedef struct {
    int u;      // source vertex
    int v;      // destination vertex
    int weight; // edge cost
} Edge;

typedef struct {
    Edge *array;
    int capacity;
    int size;
} MinHeap;

MinHeap *createMinHeap(int capacity) {
    MinHeap *minHeap = (MinHeap *)malloc(sizeof(MinHeap));
    minHeap->capacity = capacity;
    minHeap->size = 0;
    minHeap->array = (Edge *)malloc(capacity * sizeof(Edge));
    return minHeap;
}

void swapEdge(Edge *a, Edge *b) {
    Edge temp = *a;
    *a = *b;
    *b = temp;
}

void minHeapify(MinHeap *minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size &&
        minHeap->array[left].weight < minHeap->array[smallest].weight)
        smallest = left;

    if (right < minHeap->size &&
        minHeap->array[right].weight < minHeap->array[smallest].weight)
        smallest = right;

    if (smallest != idx) {
        swapEdge(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}
```

```

bool isEmpty(MinHeap *minHeap) { return minHeap->size == 0; }

Edge extractMin(MinHeap *minHeap) {
    if (isEmpty(minHeap)) {
        Edge emptyEdge = {-1, -1, -1};
        return emptyEdge;
    }

    if (minHeap->size == 1) {
        minHeap->size--;
        return minHeap->array[0];
    }

    Edge root = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    minHeap->size--;
    minHeapify(minHeap, 0);

    return root;
}

void insertMinHeap(MinHeap *minHeap, int u, int v, int weight) {
    if (minHeap->size == minHeap->capacity)
        return; // heap full

    minHeap->size++;
    int i = minHeap->size - 1;

    // insert at the bottom and bubble up
    minHeap->array[i].u = u;
    minHeap->array[i].v = v;
    minHeap->array[i].weight = weight;

    while (i != 0 &&
           minHeap->array[(i - 1) / 2].weight > minHeap->array[i].weight) {
        swapEdge(&minHeap->array[i], &minHeap->array[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

void freeMinHeap(MinHeap *minHeap) {
    free(minHeap->array);
    free(minHeap);
}

#endif
```

2. Disjoint Set Union (DisjointSetUnion.h)

```c
#ifndef DISJOINT_SET_UNION_H
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *parent;
    int *rank;
    int n;
} DisjointSet;

DisjointSet *createSet(int n) {
    DisjointSet *disjoint = (DisjointSet *)malloc(sizeof(DisjointSet));
    disjoint->n = n;
    disjoint->parent = (int *)malloc(n * sizeof(int));
    disjoint->rank = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        disjoint->parent[i] = i;
        disjoint->rank[i] = 0;
    }
    return disjoint;
}

int find(DisjointSet *ds, int i) {
    if (ds->parent[i] != i) {
        ds->parent[i] = find(ds, ds->parent[i]);
    }
    return ds->parent[i];
}

bool unionSets(DisjointSet *ds, int a, int b) {
    int rootA = find(ds, a);
    int rootB = find(ds, b);

    if (rootA != rootB) {
        if (ds->rank[rootA] < ds->rank[rootB]) {
            ds->parent[rootA] = rootB;
        } else if (ds->rank[rootA] > ds->rank[rootB]) {
            ds->parent[rootB] = rootA;
        } else {
            ds->parent[rootB] = rootA;
            ds->rank[rootA]++;
        }
        return true;
    }
    return false;
}

bool isSameSet(DisjointSet *ds, int a, int b) {
    return find(ds, a) == find(ds, b);
}

void freeSet(DisjointSet *ds) {
    free(ds->parent);
    free(ds->rank);
    free(ds);
}
```

```c

void displaySet(DisjointSet *ds) {
    int n = ds->n;

    printf("Elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", i);
    printf("\n");

    printf("Parents:  ");
    for (int i = 0; i < n; i++)
        printf("%d ", ds->parent[i]);
    printf("\n");

    printf("Ranks:    ");
    for (int i = 0; i < n; i++)
        printf("%d ", ds->rank[i]);
    printf("\n");
}

#endif // !DISJOINT_SET_UNION_H
```

3. Adjacency List (AdjacencyList.h)

```c
#ifndef ADJACENCY_LIST_H
#define ADJACENCY_LIST_H

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    struct Node *next;
    int vertex;
    int weight;
} Node;

Node **createAdjacencyList(int vertices) {
    Node **list = (Node **)malloc(vertices * sizeof(Node *));
    for (int i = 0; i < vertices; i++) {
        list[i] = NULL;
    }
    return list;
}

bool addEdgeAdjacencyListDirectedWithoutWeight(Node **list, int u, int v) {
    if (list == NULL)
        return false;
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = 0;
    newNode->next = list[u];
    list[u] = newNode;

    return true;
}

bool addEdgeAdjacencyListUnDirectedWithoutWeight(Node **list, int u, int v) {
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, u, v))
        return false;
    if (!addEdgeAdjacencyListDirectedWithoutWeight(list, v, u))
        return false;
    return true;
}

bool addEdgeAdjacencyListDirectedWithWeight(Node **list, int u, int v,
                                            int weight) {
    if (list == NULL)
        return false;

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (!newNode)
        return false;

    newNode->vertex = v;
    newNode->weight = weight;
    newNode->next = list[u];
    list[u] = newNode;

```

```c
    return true;
}

bool addEdgeAdjacencyListUnDirectedWithWeight(Node **list, int u, int v,
                                              int weight) {
    if (!addEdgeAdjacencyListDirectedWithWeight(list, u, v, weight))
        return false;
    if (!addEdgeAdjacencyListDirectedWithWeight(list, v, u, weight))
        return false;
    return true;
}

void displayAdjacencyList(Node **list, int vertices) {
    printf("\nAdjacency List Representation\n");
    printf("------------------------------\n");
    for (int i = 0; i < vertices; i++) {
        printf("Vertex [%d]: ", i);
        Node *temp = list[i];

        while (temp != NULL) {
            printf("%d(w:%d) -> ", temp->vertex, temp->weight);
            temp = temp->next;
        }
        printf("NULL\n");
    }
    printf("------------------------------------\n");
}

void freeAdjacencyList(Node **list, int vertices) {
    if (list == NULL)
        return;

    for (int i = 0; i < vertices; i++) {
        Node *current = list[i];
        while (current != NULL) {
            Node *temp = current;
            current = current->next;
            free(temp);
        }
    }
    free(list);
}

#endif // ADJACENCY_LIST_H
```